DataGIS

PLATAFORMA GEOESPACIAL

Manual de Instalação

Guia de instalação, configuração e verificação do ambiente (backend Spring Boot, frontend Vue.js, Oracle, Keycloak e GeoServer)

Versão do documento: 2.0
Data: 24/10/2026
Projeto: GeoRural DataHub – FATEC / Visiona Tecnologia Espacial
Equipe: LizardsDBA

Sumário

1. Introdução 4
1.1. Objetivo do documento 4
1.2. Público-alvo 4
1.3. Escopo 4
1.4. O que mudou na versão 2.0 4
1.5. Convenções utilizadas 4
2. Visão Geral da Solução 5
2.1. Componentes 5
2.2. Organização do banco de dados 5
2.3. Fluxo de autenticação 6
3. Requisitos 7
3.1. Hardware recomendado 7
3.2. Software 7
4. Preparação do Ambiente 9
4.1. Usuários de Windows (WSL2) 9
4.2. Linux e macOS 9
5. Acesso à Imagem do Oracle (conta, licença e login) 10
5.1. Criar a conta Oracle 10
5.2. Aceitar a licença no Container Registry 10
5.3. Gerar o Auth Token e fazer login no Docker 10
6. Obtenção do Código-Fonte 11
7. Configuração das Variáveis de Ambiente 12
7.1. Propriedades da aplicação (backend) 13
8. Instalação com Docker Compose (recomendada) 14
8.1. Primeira execução 14
8.2. O que acontece na inicialização 14
8.3. Primeiro acesso e usuários 15
9. Instalação sem Contêineres para a Aplicação 17
9.1. Serviços de apoio 17
9.2. Backend 17
9.3. Frontend 17
9.4. Dev Container (VS Code) 17
10. Verificação da Instalação 18
10.1. Obter um token para testes de API 18
10.2. Principais endpoints da API 19
11. Execução dos Testes Automatizados 21
11.1. Backend 21
11.2. Frontend 21
12. Operação do Ambiente 22
12.1. Comandos do dia a dia 22
12.2. Atualização de versão 22
12.3. Recriar o banco do zero 22
12.4. Backup e restauração 23
13. Ambiente Publicado (HTTPS) 24
13.1. O que muda em relação ao desenvolvimento 24

13.2. Subida....24
13.3. Configuração pós-instalação do Keycloak....24
13.4. Geração de pacotes sem Docker....25
Backend (arquivo .jar)....25
Frontend (arquivos estáticos)....25
14. Solução de Problemas....27
15. Referências....29
16. Controle de Versões....30

1. Introdução

1.1. Objetivo do documento

Este manual descreve, passo a passo, como preparar a máquina, obter o código-fonte, configurar e colocar em funcionamento a plataforma DataGIS, composta por um backend em Java/Spring Boot, um frontend em Vue.js, um banco de dados Oracle com suporte espacial (Oracle Spatial), um provedor de identidade (Keycloak) e um servidor de mapas (GeoServer).

1.2. Público-alvo

Desenvolvedores, administradores de sistema e integrantes da equipe técnica responsáveis por instalar, manter ou atualizar o ambiente. Pressupõe-se familiaridade básica com terminal Linux, Git e Docker.

1.3. Escopo

O documento cobre a instalação em ambiente de desenvolvimento/homologação local (cinco serviços orquestrados via Docker Compose, mais um contêiner auxiliar de configuração do GeoServer) e o ambiente publicado em servidor, que acrescenta o proxy reverso Caddy com HTTPS. Também apresenta uma alternativa de execução sem contêineres para o backend e o frontend.

1.4. O que mudou na versão 2.0

Esta versão acompanha o código da Sprint 2. As principais mudanças em relação à versão 1.0 são:

- **Autenticação e controle de acesso**: a API deixou de ser aberta. O login é feito pelo Keycloak e toda rota exige um token com as permissões do perfil do usuário (seção 8.3).
- **Novo banco**: a imagem do Oracle passou de `gvenzl/oracle-free` para a Oracle Database Enterprise (Oracle 26ai), o que exige conta Oracle e login no Oracle Container Registry (seção 5).
- **Novos serviços**: Keycloak (porta 8180) e GeoServer (porta 8085), além do contêiner `geoserver-config`, que cadastra as camadas de mapa automaticamente.
- **Novos schemas e migrations**: o schema QUARTZ passou a ser usado pelo agendador e as migrations Flyway chegaram à V082 (77 arquivos). A pasta `seed-demo` deixou de existir.
- **Ambiente publicado**: novo arquivo `docker-compose.prod.yml`, com Caddy, HTTPS e roteamento por caminho (seção 13).
- **Novos módulos na interface**: Indicadores e Resultados (mapa e tabela), Logs de Auditoria e Controle de Acesso, descritos no Manual do Usuário.

1.5. Convenções utilizadas

Convenção	Significado
`texto em fonte monoespaçada`	Comandos, nomes de arquivos, variáveis ou valores a digitar exatamente como mostrado.
Caixa de código	Bloco de comandos a ser executado no terminal, a partir da pasta indicada.
NOTA / DICA	Informação complementar que facilita a instalação.
ATENÇÃO / IMPORTANTE	Ponto que, se ignorado, pode causar falha ou risco de segurança.

2. Visão Geral da Solução

A DataGIS centraliza, trata e dá rastreabilidade a dados socioambientais de imóveis rurais (CAR, embargos, unidades de conservação, desmatamento, focos de calor, entre outros), organizando-os em uma arquitetura de dados em camadas (Medalhão): Bronze (dado bruto), Silver (dado tratado), Gold (indicadores publicados) e Quarentena (registros reprovados na validação). A partir da Sprint 2, os indicadores da camada Gold alimentam um mapa e uma tabela de resultados por município e por imóvel.

2.1. Componentes

Componente	Tecnologia	Porta padrão	Função
Banco de dados	Oracle Database Enterprise 26ai (imagem `container-registry.oracle.com/database/enterprise`, fixada por digest), com Oracle Spatial; PDB `FREEPDB1`	1521	Armazena todas as camadas em schemas separados.
Backend	Java 21, Spring Boot 4.1.1, Hibernate Spatial, Flyway, Quartz, GeoTools 35.1, JTS 1.20, springdoc 3.0.3	8080	API REST protegida por token, ingestão de fontes, validação, transição Bronze → Silver → Gold, publicação de indicadores, auditoria e administração de usuários.
Frontend	Vue 3, Vite 8, Vue Router 5, Leaflet, keycloak-js 26, jsPDF	5173	Interface web usada por operadores, analistas, auditores e administradores.
Keycloak	Keycloak 26.8.0 (modo `start-dev`, volume `keycloak-data`), realm `datagis`	8180	Login, usuários, perfis e permissões (RBAC).
GeoServer	GeoServer 3.0.2 com extensões Oracle e QueryLayer	8085 (somente 127.0.0.1)	Publica as camadas de mapa (WMS/WFS) a partir das views do schema GOLD. Acessado apenas pelo proxy do backend.
geoserver-config	Contêiner `curl` que executa `infra/geoserver/configurar.sh`	–	Cria workspace, datastore, estilos e camadas no GeoServer. Termina sozinho.
Caddy (somente no ambiente publicado)	Caddy 2.10.2	80 / 443	HTTPS automático e roteamento por caminho.

2.2. Organização do banco de dados

É utilizada uma única instância Oracle (PDB FREEPDB1) com um schema por camada. Apenas o usuário CONTROLE é usado para conexão pelo backend; os demais schemas existem somente como donos de tabelas.

Schema	Conteúdo
CONTROLE	Catálogo de fontes, lotes de ingestão, mapeamento de colunas (de-para), conjuntos, trilha de auditoria (`log_auditoria`). Usuário de conexão da aplicação.
BRONZE	Tabela `registro_bruto` com o payload original em JSON (schema-on-read).
SILVER	Entidades canônicas: imóvel rural, área protegida, embargo, auto de infração, desmatamento, foco de calor, município e tabelas de referência.
GOLD	Tipos de indicador, resultados por imóvel e município, camadas de mapa publicadas (views `vw_mapa_*`) e detalhe congelado por imóvel.
QUARENTENA	Tabelas espelho `q_*` com registros reprovados e motivo da rejeição.
QUARTZ	Tabelas do agendador Quartz (execução agendada das fontes).

A criação das tabelas e a carga de dados iniciais são feitas automaticamente pelo Flyway na primeira subida do backend, a partir dos scripts em `src/main/resources/db/migration`.

2.3. Fluxo de autenticação

1. O navegador abre o frontend (`http://localhost:5173`). Sem sessão, o `keycloak-js` redireciona para a tela de login do Keycloak (`http://localhost:8180`).
2. Após o login, o Keycloak devolve um token de acesso (validade de 5 minutos, renovado automaticamente pelo frontend).
3. O frontend chama `/api/...` com o cabeçalho `Authorization: Bearer <token>`. O Vite (em desenvolvimento) ou o Caddy (no servidor) encaminha a chamada ao backend.
4. O backend valida a assinatura do token e libera ou nega cada rota conforme as permissões do usuário (seção 10.2).

3. Requisitos

3.1. Hardware recomendado

Os valores abaixo são **estimativas da equipe de documentação** (não medições formais) e consideram a imagem Enterprise do Oracle, o Keycloak, o GeoServer e o backend com até 8 GB de heap (`-Xmx8g`), que são mais pesados que os da versão 1.0.

Recurso	Mínimo	Recomendado	Observação
Memória RAM	12 GB	16 GB ou mais	O Oracle Enterprise, o GeoServer e o Keycloak somam vários GB além do backend (`-Xmx8g`). No Docker Desktop/WSL2, libere ao menos 12 GB para o Docker.
Processador	4 núcleos	8 núcleos	A primeira compilação Maven, a ingestão de arquivos grandes e o cálculo dos indicadores são intensivos.
Disco livre	30 GB	60 GB ou mais	Imagens Docker (a do Oracle Enterprise tem vários GB), dependências Maven/npm, datafiles do Oracle e arquivos enviados (shapefiles do CAR podem ultrapassar 3 GB).
Rede	Acesso à internet	Banda larga estável	Necessária para baixar imagens e dependências e para ingerir fontes remotas (WFS / ArcGIS REST).

3.2. Software

Software	Versão	Obrigatório?	Uso
Sistema operacional	Linux (Ubuntu 22.04+), macOS ou Windows 10/11 com WSL2	Sim	A equipe desenvolve em Ubuntu dentro do WSL2.
Git	2.30 ou superior	Sim	Obtenção do código-fonte.
Docker Engine / Docker Desktop	24 ou superior, com Docker Compose v2	Sim	Execução de todos os serviços.
Conta Oracle	Conta gratuita (não é conta de nuvem/OCI)	Sim	Necessária para baixar a imagem Oracle Enterprise (seção 5).
Java JDK	21 (ex.: Eclipse Temurin 21)	Somente para execução sem Docker	Compilar e executar o backend na máquina.
Node.js / npm	Node 22 LTS	Somente para execução sem Docker	Executar o frontend na máquina.
curl	Qualquer versão recente	Somente para execução sem Docker	O backend usa o curl do sistema para acessar alguns serviços externos (ex.: WFS do SICAR).
Navegador	Chrome, Edge ou Firefox atualizados	Sim	Acesso à interface.

Software	Versão	Obrigatório?	Uso
IDE (opcional)	VS Code ou IntelliJ IDEA	Não	O projeto inclui configuração de Dev Container para VS Code.

> **NOTA:** O Maven não precisa ser instalado: o projeto utiliza o Maven Wrapper (`./mvnw`), que baixa a versão correta automaticamente.

4. Preparação do Ambiente

4.1. Usuários de Windows (WSL2)

Em Windows, recomenda-se executar todo o projeto dentro do WSL2 com Ubuntu, com o Docker Desktop configurado para integração com o WSL.

1. Instale o WSL2 e o Ubuntu executando `wsl --install` em um PowerShell como administrador e reinicie o computador.
2. Instale o Docker Desktop e, em Settings > Resources > WSL Integration, habilite a integração com a distribuição Ubuntu. Em Resources, reserve memória suficiente (seção 3.1).
3. Abra o terminal do Ubuntu e confirme o Docker com `docker version` e `docker compose version`.
4. Mantenha o código-fonte no sistema de arquivos do Linux (ex.: `~/projetos`), e não em `/mnt/c/...`, pois o acesso a pastas do Windows a partir de contêineres é muito lento.
5. Configure o Git para não converter finais de linha, evitando erros do tipo "bad interpreter" em scripts `.sh`:

```
git config --global core.autocrlf input
```

4.2. Linux e macOS

Instale o Docker Engine (ou Docker Desktop no macOS) e o Git pelos canais oficiais. No Linux, adicione seu usuário ao grupo docker para evitar o uso de sudo:

```
sudo usermod -aG docker $USER
# encerre a sessão e entre novamente para aplicar
```

5. Acesso à Imagem do Oracle (conta, licença e login)

A imagem `gvenzl/oracle-free`, usada na versão 1.0, tem um limite de 12 GB de dados embutido na licença. A versão 2.0 usa a Oracle Database Enterprise, distribuída gratuitamente pela Oracle para desenvolvimento e teste, mas que só pode ser baixada após criar uma conta, aceitar a licença e fazer login no Oracle Container Registry. Este procedimento é feito **uma única vez por máquina**.

> **IMPORTANTE:** A licença de desenvolvedor da Oracle permite uso **somente em desenvolvimento e teste**. Antes de usar o banco em um ambiente de produção de verdade, a equipe deve avaliar o licenciamento adequado junto à Oracle.

5.1. Criar a conta Oracle

1. Acesse `https://profile.oracle.com/myprofile/account/create-account.jspx` (é a conta simples da Oracle, **não** a conta de nuvem OCI).
2. Preencha e-mail, nome e país. A senha exige no mínimo 8 caracteres, com letra minúscula, maiúscula, número e caractere especial.
3. Confirme o e-mail recebido (o link expira em cerca de 30 minutos). Quem já possui conta Oracle pode usá-la.

5.2. Aceitar a licença no Container Registry

1. Acesse `https://container-registry.oracle.com` e faça login com a conta criada.
2. Procure por "Database" e entre no repositório **enterprise**.
3. Clique em **Continue**, leia os termos ("Oracle Standard Terms and Restrictions") e clique em **Accept**.
4. Confirme que a página passou a exibir a data em que você aceitou os termos. Sem essa confirmação o download será negado.

5.3. Gerar o Auth Token e fazer login no Docker

1. Ainda logado no Container Registry, abra o menu da sua conta (canto superior direito), escolha **Auth Token** e clique em **Generate Key**.
2. Copie e guarde o token imediatamente: ele não pode ser exibido novamente.
3. No terminal, execute o login. Como usuário informe o e-mail da conta Oracle e, como senha, o **Auth Token** (não a senha da conta):

```
docker login container-registry.oracle.com
```

O resultado esperado é `Login Succeeded`. Em caso de `unauthorized` ou `denied`, confira se o token foi copiado sem espaços e se o e-mail é o mesmo da conta que aceitou a licença.

> **DICA:** Para testar antes da instalação completa, execute `docker pull container-registry.oracle.com/database/enterprise:latest`. Se baixar sem erro de permissão, o acesso está correto (o download tem vários GB).

6. Obtenção do Código-Fonte

O backend e o frontend são mantidos em repositórios Git separados. O frontend deve ficar dentro da pasta do backend, com o nome `datagis-frontend`, pois é esse o caminho referenciado pelo `docker-compose.yml`.

```
mkdir -p ~/projetos && cd ~/projetos
git clone git@github.com:LizardsDBA/datagis-backend.git
cd datagis-backend
git clone git@github.com:LizardsDBA/datagis-frontend.git datagis-frontend
```

Caso o código tenha sido recebido em arquivo compactado (.zip), basta extraí-lo mantendo a mesma estrutura. A organização esperada é:

```
datagis-backend/
├── docker-compose.yml          # oracle + keycloak + geoserver + backend + frontend
├── docker-compose.prod.yml     # override do ambiente publicado (Caddy, HTTPS)
├── Dockerfile.dev              # imagem de desenvolvimento do backend
├── .env.example                # modelo de variáveis de ambiente
├── mvnw / mvnw.cmd / pom.xml   # build Maven do backend
├── infra/
│   ├── oracle-init/            # cria os schemas e concede privilégios no Oracle
│   ├── keycloak/               # realm-datagis.json (clients, perfis, permissões)
│   ├── geoserver/              # configurar.sh e estilos das camadas de mapa
│   └── caddy/Caddyfile         # proxy do ambiente publicado
├── src/main/java/...           # código do backend
├── src/main/resources/         # application*.properties e migrations Flyway
├── docs/                       # documentação técnica complementar
└── datagis-frontend/           # aplicação Vue.js
    ├── Dockerfile.dev
    ├── package.json / vite.config.js
    └── src/
```

Garanta que o Maven Wrapper tenha permissão de execução (necessário quando o código veio de uma máquina Windows):

```
chmod +x mvnw
```

7. Configuração das Variáveis de Ambiente

As credenciais e as portas expostas são lidas do arquivo `.env`, na raiz de `datagis-backend`. Crie-o a partir do modelo:

```
cd ~/projetos/datagis-backend
cp .env.example .env
```

Edite o arquivo e defina senhas próprias:

Variável	Valor do modelo	Descrição
`ORACLE_PASSWORD`	troque_a_senha_sys	Senha dos usuários administrativos SYS e SYSTEM do Oracle.
`APP_USER`	CONTROLE	Usuário/schema de conexão da aplicação. Mantenha CONTROLE, pois as migrations dependem desse nome.
`APP_USER_PASSWORD`	troque_a_senha_app	Senha do usuário CONTROLE, usada pelo backend e pelo GeoServer.
`SCHEMA_PASSWORD`	troque_a_senha_schemas	Senha interna dos schemas BRONZE, SILVER, GOLD, QUARENTENA, QUARTZ e BATCH.
`ORACLE_PORT`	1521	Porta do Oracle exposta na máquina.
`BACKEND_PORT`	8080	Porta da API exposta na máquina.
`FRONTEND_PORT`	5173	Porta da interface web exposta na máquina.
`KEYCLOAK_PORT`	8180	Porta do Keycloak (login e console administrativo).
`KEYCLOAK_ADMIN_PASSWORD`	troque_a_senha_keycloak	Senha do usuário `admin` do console do Keycloak.
`KEYCLOAK_BACKEND_CLIENT_SECRET`	datagis-backend-dev-secret	Segredo do client `datagis-backend`. Deve ser igual ao definido em `infra/keycloak/realm-datagis.json`.
`GEOSERVER_ADMIN_PASSWORD`	troque_a_senha_geoserver	Senha do usuário `admin` do GeoServer.
`GEOSERVER_PORT`	8085	Porta do GeoServer, publicada apenas em 127.0.0.1.

IMPORTANTE: O repositório já contém um arquivo `.env` com senhas de desenvolvimento, por decisão da equipe. Essas credenciais só são aceitáveis em uma máquina local. Em qualquer ambiente acessível por outras pessoas (servidor de homologação, nuvem), defina senhas novas, inclusive o segredo do client do Keycloak, e não reutilize as do repositório.

As senhas do Oracle só são aplicadas na primeira criação do banco. Para trocá-las depois, é preciso recriar o volume (seção 12.3), o que apaga os dados.

NOTA: O comentário do `.env.example` ainda menciona a imagem `gvenzl/oracle-free`. Ele está desatualizado: a imagem em uso é a Oracle Enterprise descrita na seção 5.

7.1. Propriedades da aplicação (backend)

As demais configurações ficam em `src/main/resources/application.properties` e normalmente não precisam ser alteradas:

Propriedade	Valor	Descrição
`spring.flyway.schemas`	CONTROLE, BRONZE, SILVER, GOLD, QUARENTENA, QUARTZ	Schemas gerenciados pelo Flyway.
`spring.flyway.locations`	controle, bronze, silver, gold, quarentena, quartz	Pastas de migrations aplicadas. A antiga pasta `seed-demo` (dados fictícios) não existe mais.
`spring.jpa.hibernate.ddl-auto`	none	O Flyway é o único responsável pela estrutura do banco.
`spring.servlet.multipart.max-file-size`	-1 (sem limite)	Permite o envio de arquivos grandes (shapefiles do CAR).
`app.armazenamento.diretorio`	/tmp/datagis-uploads	Pasta temporária dos arquivos enviados por upload.
`app.keycloak.url-publica`	`KEYCLOAK_URL_PUBLICA` (padrão http://localhost:8180)	URL do Keycloak vista pelo navegador. Deve coincidir com o emissor (`iss`) dos tokens.
`app.keycloak.url-interna`	`KEYCLOAK_URL_INTERNA` (padrão http://localhost:8180)	URL usada pelo backend para falar com o Keycloak. No Compose é http://keycloak:8080.
`app.keycloak.realm` / `client-id`	datagis / datagis-backend	Realm e client usados pelo backend.
`app.keycloak.provisionar`	`true` no perfil dev/docker	Sincroniza perfis e permissões no Keycloak a cada subida do backend.
`app.keycloak.usuarios-dev`	`true` no perfil dev/docker; `false` no servidor	Cria os usuários de teste (seção 8.3).

8. Instalação com Docker Compose (recomendada)

Este é o método padrão. O Docker Compose sobe os serviços `oracle`, `keycloak`, `geoserver`, `geoserver-config`, `backend` e `frontend`. O backend e o frontend rodam em modo de desenvolvimento, com o código-fonte montado a partir da máquina, de modo que alterações no código são refletidas sem reconstruir as imagens.

8.1. Primeira execução

1. Conclua o login no Oracle Container Registry (seção 5) e crie o `.env` (seção 7).
2. Acesse a pasta do projeto e suba os serviços em segundo plano:

```
cd ~/projetos/datagis-backend
docker compose up -d --build
```

1. Acompanhe a inicialização. O Oracle precisa ficar `healthy` antes de o backend iniciar (o Compose aguarda automaticamente):

```
docker compose ps
docker compose logs -f oracle      # Ctrl+C para sair
docker compose logs -f backend
```

1. Na primeira subida, o backend baixa as dependências Maven, o Flyway cria as tabelas e o provisionador sincroniza perfis e permissões no Keycloak. Aguarde mensagens semelhantes a:

```
Successfully applied NN migrations to schema "CONTROLE", now at version v082
Started DatagisApplication in XX seconds
```

1. O frontend executa `npm install` e depois inicia o Vite. Aguarde a mensagem `VITE ... ready` (`docker compose logs -f frontend`).
2. O contêiner `geoserver-config` configura o GeoServer e encerra com status `Exited (0)`. Isso é esperado.
3. Acesse `http://localhost:5173`. Você será redirecionado ao login do Keycloak (seção 8.3).

> **NOTA:** A primeira execução pode levar de 10 a 30 minutos, dependendo da conexão, por causa do download da imagem do Oracle Enterprise (vários GB) e das dependências. As execuções seguintes levam poucos minutos, pois os volumes `maven-repo`, `frontend-node-modules`, `oracle-data` e `keycloak-data` guardam cache e dados.

8.2. O que acontece na inicialização

Etapa	Responsável	Descrição
1	Contêiner `oracle`	Cria o banco e o usuário CONTROLE (variáveis `APP_USER`/`APP_USER_PASSWORD`).

Etapa	Responsável	Descrição
2	`infra/oracle-init/01-criar-schemas.sh`	Executado uma única vez, com o banco vazio: ajusta `PGA_AGGREGATE_LIMIT` para 8 GB, cria os schemas BRONZE, SILVER, GOLD, QUARENTENA, QUARTZ e BATCH e concede privilégios ao CONTROLE (inclusive os necessários para índices espaciais).
3	Contêiner `keycloak`	Sobe em `start-dev` e importa o realm `datagis` (clients, perfis, permissões).
4	Contêiner `geoserver`	Sobe o GeoServer 3.0.2 com as extensões Oracle e QueryLayer.
5	Contêiner `backend`	Executa `./mvnw spring-boot:run` com o perfil `docker`, conectando em `oracle:1521/FREEPDB1`.
6	Flyway	Aplica as migrations: tabelas de controle, Bronze, Silver, Gold, Quarentena, Quartz, tabelas de referência, fontes pré-cadastradas, tipos de indicador e pacotes PL/SQL de cálculo.
7	Provisionador do Keycloak	Sincroniza as 13 permissões e os 4 perfis no Keycloak e, em desenvolvimento, cria os usuários de teste.
8	Contêiner `geoserver-config`	Executa `infra/geoserver/configurar.sh`: cria workspace, datastore Oracle, estilos e camadas das views do schema GOLD. Para repetir: `docker compose run --rm geoserver-config`.
9	Contêiner `frontend`	Executa `npm install` e `npm run dev`. O Vite encaminha `/api` e `/conjuntos` ao backend, evitando problemas de CORS.

8.3. Primeiro acesso e usuários

Em desenvolvimento, o backend cria quatro usuários de teste, um por perfil. **A senha inicial de cada um é igual ao próprio login** e o Keycloak exige a troca no primeiro acesso:

Login	Perfil	O que pode fazer
`admin`	Administrador	Todas as 13 permissões, inclusive administração de usuários.
`operador`	Operador	Fontes, lotes e quarentena (visualizar e gerenciar) e consulta aos dados.
`analista`	Analista	Conjuntos e indicadores (visualizar e gerenciar/publicar), consulta a fontes e dados.
`auditor`	Auditor	Somente leitura em todas as telas e nos logs de auditoria.

> **ATENÇÃO:** Esses usuários existem apenas quando `app.keycloak.usuarios-dev` é `true`. No ambiente publicado essa opção fica desligada e o primeiro administrador é criado pelo console do Keycloak (seção 13.3).

O console administrativo do Keycloak fica em `http://localhost:8180` (usuário `admin`, senha definida em `KEYCLOAK_ADMIN_PASSWORD`). O GeoServer fica em `http://localhost:8085/geoserver` (usuário `admin`, senha definida em `GEOSERVER_ADMIN_PASSWORD`) e só é acessível a partir da própria máquina.

> **IMPORTANTE:** O realm versionado aceita apenas `http://localhost:5173/*` como endereço de

retorno do login. Se você alterar `FRONTEND_PORT` ou `KEYCLOAK_PORT`, o login passa a falhar (`Invalid parameter: redirect_uri`) até que o client `datagis-frontend` seja atualizado no Keycloak (console > Clients > datagis-frontend, campos Valid redirect URIs e Web origins) ou o arquivo `infra/keycloak/realm-datagis.json` seja ajustado antes da primeira subida.

9. Instalação sem Contêineres para a Aplicação

Esta alternativa é útil para depurar o backend em uma IDE. Oracle, Keycloak e GeoServer continuam rodando em contêiner; backend e frontend rodam diretamente na máquina.

9.1. Serviços de apoio

```
cd ~/projetos/datagis-backend
docker compose up -d oracle keycloak geoserver geoserver-config
docker compose ps    # aguarde "healthy" no oracle
```

9.2. Backend

Use o perfil `dev`, que aponta para `localhost`. As credenciais são lidas de variáveis de ambiente:

```
cd ~/projetos/datagis-backend
export APP_USER=CONTROLE
export APP_USER_PASSWORD=<senha definida no .env>
export KEYCLOAK_BACKEND_CLIENT_SECRET=<valor definido no .env>
export SPRING_PROFILES_ACTIVE=dev
./mvnw spring-boot:run
```

Na IDE, configure a classe principal `br.gov.sp.cps.datagis.DatagisApplication`, o perfil ativo `dev` e as mesmas variáveis de ambiente. Se o GeoServer não estiver em `http://localhost:8085/geoserver`, informe a URL em `GEOSERVER_URL`.

9.3. Frontend

O proxy do Vite aponta para o host `backend`, que só existe dentro da rede do Docker. Ao executar o frontend fora do contêiner, altere temporariamente o alvo em `datagis-frontend/vite.config.js` para `http://localhost:8080` nas entradas `/api` e `/conjuntos` (não versione essa alteração):

```
proxy: {
  '/api': { target: 'http://localhost:8080', changeOrigin: true },
  '/conjuntos': { target: 'http://localhost:8080', changeOrigin: true },
},
```

```
cd ~/projetos/datagis-backend/datagis-frontend
npm install
npm run dev
```

A interface ficará disponível em `http://localhost:5173`, com login pelo Keycloak em `http://localhost:8180`.

9.4. Dev Container (VS Code)

O projeto inclui `.devcontainer/devcontainer.json`, baseado no `Dockerfile.dev` do backend. Com a extensão Dev Containers instalada, use o comando Dev Containers: Reopen in Container para abrir o projeto já com o JDK 21 configurado. Oracle, Keycloak e GeoServer devem continuar sendo iniciados pelo Docker Compose.

10. Verificação da Instalação

Após a subida, confirme o funcionamento de cada componente:

Verificação	Como fazer	Resultado esperado
Contêineres ativos	`docker compose ps`	`oracle` (healthy), `keycloak`, `geoserver`, `backend` e `frontend` em execução; `geoserver-config` encerrado com código 0.
Saúde do backend	`curl http://localhost:8080/actuator/health`	`{"status":"UP"}`
Login (Keycloak)	Abrir `http://localhost:8180`	Página do Keycloak; o console aceita o usuário `admin`.
Interface web	Abrir `http://localhost:5173`	Redirecionamento à tela "Entrar na sua conta"; após o login, a tela "Fontes de Dados" com o menu lateral DataGIS.
API protegida	`curl -i http://localhost:8080/api/fontes`	Resposta `401 Unauthorized`, pois a rota exige token (comportamento correto).
API com token	Obter um token (seção 10.1) e chamar `curl -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/fontes`	Lista em JSON com as fontes pré-cadastradas.
Documentação da API	Abrir `http://localhost:8080/swagger-ui.html` e usar o botão Authorize	Swagger UI com os endpoints; as chamadas funcionam após informar o token.
GeoServer	Abrir `http://localhost:8085/geoserver`	Painel do GeoServer; após publicar indicadores, as camadas `vw_mapa_*` aparecem no workspace do DataGIS.
Banco de dados	Conectar com SQL Developer/DBeaver em `localhost:1521`, serviço `FREEPDB1`, usuário `CONTROLE`	Conexão aceita e schemas visíveis.

10.1. Obter um token para testes de API

Em desenvolvimento existe o client `datagis-dev`, que permite obter um token pelo fluxo de senha. O exemplo abaixo usa o usuário `admin` (informe a senha que você definiu após o primeiro acesso):

```
TOKEN=$(curl -s -X POST
http://localhost:8180/realms/datagis/protocol/openid-connect/token \
  -d grant_type=password -d client_id=datagis-dev \
  -d username=admin -d password=<senha> | python3 -c "import
sys,json;print(json.load(sys.stdin)['access_token'])")
```

> **NOTA:** O token expira em 5 minutos. No Swagger UI, clique em **Authorize**, cole o token e confirme.

10.2. Principais endpoints da API

Todas as rotas abaixo (exceto `/actuator/health`, `/api/monitoramento/**` e o Swagger) exigem token e a permissão indicada.

Método e rota	Finalidade	Permissão
GET / POST `/api/fontes`	Listar e cadastrar fontes de dados.	`fonte:visualizar` / `fonte:gerenciar`
GET / PUT `/api/fontes/{id}`	Consultar e editar uma fonte.	idem
POST `/api/fontes/{id}/executar`	Disparar a ingestão de uma fonte remota.	`fonte:gerenciar`
POST `/api/fontes/{id}/upload`	Enviar arquivo (multipart) de uma fonte de upload manual.	`fonte:gerenciar`
GET `/api/fontes/{id}/progresso`	Acompanhar o progresso da execução.	`fonte:visualizar`
GET / POST / PATCH / DELETE `/api/fontes/{fonteId}/mapeamentos`	Gerenciar o de-para de colunas Bronze → Silver e a tabela Silver de destino.	`fonte:*`
GET `/api/lotes`, `/api/lotes/{id}`	Histórico de lotes de ingestão.	`lote:visualizar`
POST `/api/quarentena/lotes/{loteId}/(liberar | descartar)`	Liberar para a Silver ou descartar um lote retido.	`lote:gerenciar`
GET `/api/quarentena`, `/metricas`, `/tipos-erro`, `/status-analise`	Consultas da quarentena.	`quarentena:visualizar`
POST `/api/quarentena/{entidade}/{id}/(iniciar-analise | descartar | reprocessar)`	Ações sobre um registro retido.	`quarentena:tratar`
GET `/api/conjuntos-dados/...`	Comparação Bronze x Silver x Quarentena e diferença entre lotes.	`dados:visualizar`
GET / POST `/conjuntos`	Listar e cadastrar conjuntos.	`conjunto:visualizar` / `conjunto:gerenciar`
GET `/api/indicadores/...`, `/api/gold/...`	Resultados por município e imóvel, status de publicação, proxy WMS/WFS do GeoServer (`/api/indicadores/geoserver/{wms|wfs}`).	`indicador:visualizar`

Método e rota	Finalidade	Permissão
POST `/api/gold/publicar`, `/api/indicadores/...`	Migrar para Gold (calcular e publicar indicadores), recalcular, gerenciar tipos de indicador.	`indicador:publicar`
GET `/api/auditoria`	Consulta paginada da trilha de auditoria.	`auditoria:visualizar`
`/api/admin/usuarios`, `/api/admin/catalogo-acesso`	Cadastro de usuários, perfis, redefinição de senha e catálogo de acesso.	`usuario:gerenciar`

11. Execução dos Testes Automatizados

11.1. Backend

```
# dentro do contêiner
docker compose exec backend ./mvnw test

# ou na máquina (JDK 21 instalado)
./mvnw test
```

> **NOTA:** Os testes de integração que usam Testcontainers (por exemplo `DatagisApplicationTests` e `ConjuntoOracleIT`) precisam de acesso ao Docker e podem falhar quando executados de dentro do contêiner do backend. Para rodá-los, execute `./mvnw test` diretamente na máquina, com o Docker disponível. Para rodar uma classe específica: `./mvnw test -Dtest=NomeDaClasseTest`.

11.2. Frontend

```
cd datagis-frontend
npm test            # execução única (Vitest)
npm run test:watch  # modo observação
```

12. Operação do Ambiente

12.1. Comandos do dia a dia

Ação	Comando
Subir todos os serviços	`docker compose up -d`
Subir após alterar Dockerfile	`docker compose up -d --build`
Parar mantendo os dados	`docker compose down`
Reiniciar somente o backend	`docker compose restart backend`
Ver logs em tempo real	`docker compose logs -f backend` (ou `oracle` / `frontend` / `keycloak` / `geoserver`)
Abrir terminal no backend	`docker compose exec backend bash`
Reaplicar a configuração do GeoServer	`docker compose run --rm geoserver-config`
Acessar o SQL*Plus	`docker compose exec oracle sqlplus CONTROLE/<senha>@FREEPDB1`

12.2. Atualização de versão

1. Atualize o código nos dois repositórios com `git pull` (backend e `datagis-frontend`).
2. Execute `docker compose up -d --build`. Novas migrations do Flyway são aplicadas automaticamente na subida do backend; novas dependências do frontend são instaladas automaticamente.
3. Confira os logs do backend para garantir que as migrations foram aplicadas sem erro e que o provisionador do Keycloak concluiu a sincronização de perfis.
4. Se a versão trouxe novas camadas de mapa, reexecute `docker compose run --rm geoserver-config`.

> **ATENÇÃO:** Nunca altere o conteúdo de uma migration já aplicada. O Flyway detecta a mudança de checksum e impede a subida do backend (`FlywayValidateException`). Toda alteração de banco deve ser feita em uma nova migration com o próximo número de versão.

12.3. Recriar o banco do zero

Apaga todos os dados e recria os schemas. Útil ao alterar senhas do `.env`, o script de inicialização ou após editar migrations em ambiente de desenvolvimento:

```
docker compose down -v
docker compose up -d
```

> **IMPORTANTE:** O parâmetro `-v` remove **todos** os volumes: os dados do Oracle (`oracle-data`) e também os usuários e sessões do Keycloak (`keycloak-data`). Os usuários de teste são recriados na subida; usuários cadastrados manualmente, não. Faça backup antes, se necessário.

12.4. Backup e restauração

Para um backup lógico dos schemas, utilize o Data Pump do próprio contêiner. Inclua o schema QUARTZ, que guarda os agendamentos das fontes:

```
docker compose exec oracle bash -c "expdp system/<ORACLE_PASSWORD>@FREEPDB1 \
  schemas=CONTROLE,BRONZE,SILVER,GOLD,QUARENTENA,QUARTZ \
  directory=DATA_PUMP_DIR dumpfile=datagis.dmp logfile=datagis_exp.log"

# copiar o arquivo para a máquina
docker compose cp oracle:/opt/oracle/admin/FREE/dpdump/<pasta-do-PDB>/datagis.dmp .
```

A restauração é feita com `impdp` usando os mesmos parâmetros. O caminho físico de `DATA_PUMP_DIR` pode ser confirmado com `SELECT directory_path FROM dba_directories WHERE directory_name = 'DATA_PUMP_DIR';`.

Os usuários e a configuração do Keycloak ficam no volume `keycloak-data` (banco H2 embutido). Para copiá-lo:

```
docker run --rm -v <projeto>_keycloak-data:/dados -v "$PWD":/backup alpine \
  tar czf /backup/keycloak-data.tgz -C /dados .
```

> **NOTA:** O nome real do volume é prefixado pelo nome da pasta do projeto; confirme com `docker volume ls`. Perfis e permissões são recriados automaticamente pelo backend, mas os **usuários** cadastrados só podem ser recuperados a partir desse backup.

13. Ambiente Publicado (HTTPS)

O arquivo `docker-compose.prod.yml` é um override do compose de desenvolvimento usado no servidor `https://datagis.com.br`. Ele adiciona o proxy Caddy, que é o único serviço com portas abertas para a internet (80 e 443), e remove a publicação de portas do Oracle, backend, frontend e Keycloak, que passam a existir apenas na rede interna do Docker.

13.1. O que muda em relação ao desenvolvimento

Item	Desenvolvimento	Ambiente publicado
Entrada	Portas 5173, 8080, 8180 e 1521 abertas na máquina	Somente Caddy (80/443), com certificado HTTPS emitido e renovado automaticamente
Keycloak	`http://localhost:8180`	`https://datagis.com.br/auth` (`KC_HTTP_RELATIVE_PATH=/auth`)
Frontend	Vite em modo dev	Compilado uma vez (`npm run build`); o Caddy serve a pasta `dist`
Usuários de teste	Criados (login = senha)	Desligados (`APP_KEYCLOAK_USUARIOS_DEV=false`)
Roteamento	Proxy do Vite	`/auth` → Keycloak, `/api` e `/conjuntos` → backend, demais caminhos → `dist` com fallback ao `index.html`

13.2. Subida

Além do `.env`, crie no servidor um arquivo `.env.prod` (fora do Git) que sobrescreve ao menos a senha do console do Keycloak, e aponte o DNS de `datagis.com.br` para o servidor antes da primeira subida, para que o Caddy consiga emitir o certificado:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml \
  --env-file .env --env-file .env.prod up -d
```

13.3. Configuração pós-instalação do Keycloak

1. O realm versionado só conhece `http://localhost:5173`. Após a primeira subida, cadastre `https://datagis.com.br/*` em Valid redirect URIs, Valid post logout redirect URIs e `https://datagis.com.br` em Web origins do client `datagis-frontend`, pelo console do Keycloak ou com `kcadm.sh`.
2. Crie o primeiro usuário administrador pelo console do Keycloak (realm `datagis`) e atribua a ele o perfil Administrador. Os demais usuários passam a ser gerenciados na tela Controle de Acesso do próprio DataGIS.
3. Troque o segredo do client `datagis-backend` (`KEYCLOAK_BACKEND_CLIENT_SECRET`) por um valor próprio.

> **ATENÇÃO:** Limitações conhecidas desta configuração, registradas para decisão da equipe:
>
> **Keycloak em `start-dev`** com banco H2 em volume: adequado para validação, mas não para produção de longa duração (sem alta disponibilidade e com risco de perda de usuários se o volume for removido).

Avalie `start` com banco externo.

Oracle Enterprise com licença de desenvolvedor: não deve ser usada em produção (seção 5).

Backend em modo dev (`./mvnw spring-boot:run`): o empacotamento em `.jar` ainda não foi adotado no compose.

13.4. Geração de pacotes sem Docker

Caso a hospedagem use outra infraestrutura, os artefatos podem ser gerados manualmente.

Backend (arquivo .jar)

```
./mvnw clean package -DskipTests
# gera target/datagis-0.0.1-SNAPSHOT.jar
```

```
export SPRING_DATASOURCE_URL=jdbc:oracle:thin:@<host-oracle>:1521/<servico>
export SPRING_DATASOURCE_USERNAME=CONTROLE
export SPRING_DATASOURCE_PASSWORD=<senha>
export KEYCLOAK_URL_PUBLICA=https://<dominio>/auth
export KEYCLOAK_URL_INTERNA=<url interna do Keycloak>
export KEYCLOAK_BACKEND_CLIENT_SECRET=<segredo>
export GEOSERVER_URL=http://<host-geoserver>:8080/geoserver
java -Xmx4g -jar target/datagis-0.0.1-SNAPSHOT.jar
```

O Oracle de destino precisa ter o Oracle Spatial disponível e os schemas criados conforme `infra/oracle-init/01-criar-schemas.sh` (incluindo os privilégios de índice espacial). O `issuer` dos tokens emitidos pelo Keycloak deve coincidir com `KEYCLOAK_URL_PUBLICA`.

Frontend (arquivos estáticos)

```
cd datagis-frontend
npm ci
VITE_KEYCLOAK_URL=https://<dominio>/auth npm run build
# gera a pasta dist/
```

Publique `dist/` em um servidor web que encaminhe `/api` e `/conjuntos` ao backend e devolva `index.html` para as demais rotas. O `infra/caddy/Caddyfile` é a referência; um equivalente em Nginx seria:

```
server {
  listen 443 ssl;
  root /var/www/datagis/dist;
  client_max_body_size 0;   # uploads grandes
  location /auth/ { proxy_pass http://127.0.0.1:8180; proxy_set_header X-Forwarded-
Proto https; proxy_set_header Host $host; }
  location /api/ { proxy_pass http://127.0.0.1:8080; proxy_read_timeout 3600s; }
  location /conjuntos { proxy_pass http://127.0.0.1:8080; }
  location / { try_files $uri $uri/ /index.html; }
}
```

IMPORTANTE: A API agora exige autenticação, mas isso não dispensa o uso de HTTPS e de firewall.

> Nunca exponha Oracle (1521), GeoServer (8085) ou o console administrativo do Keycloak diretamente à internet, e troque todas as senhas de desenvolvimento.

14. Solução de Problemas

Sintoma	Causa provável	Solução
`docker compose up` falha ao baixar a imagem do Oracle (`unauthorized` / `denied`)	Sem login no Oracle Container Registry ou licença não aceita.	Refaça a seção 5: aceite a licença do repositório `enterprise`, gere o Auth Token e execute `docker login container-registry.oracle.com`.
Backend não inicia; log mostra `ORA-00902: invalid datatype` em migrations V012+	Imagem Oracle sem o Oracle Spatial.	Use a imagem definida no compose e recrie o volume (`docker compose down -v`).
`ORA-01031` ou `ORA-29855` ao criar índice espacial	Privilégios de índice espacial não concedidos aos schemas SILVER/GOLD.	O script `01-criar-schemas.sh` só roda com o banco vazio. Recrie o volume (`docker compose down -v`) ou conceda os privilégios manualmente conforme o script.
`FlywayValidateException` / checksum mismatch	Uma migration já aplicada foi alterada.	Em desenvolvimento: `docker compose down -v`. Em ambiente com dados: reverter a alteração ou executar `flyway repair` com cautela.
`/bin/bash^M: bad interpreter` no contêiner do Oracle	Scripts `.sh` com final de linha Windows (CRLF).	Configure `git config --global core.autocrlf input`, clone novamente e recrie o volume.
`./mvnw: Permission denied`	Arquivo sem permissão de execução.	`chmod +x mvnw`.
Backend fica aguardando e não sobe	Oracle ainda não está `healthy` (primeira inicialização).	Aguarde; acompanhe com `docker compose logs -f oracle`.
`port is already allocated`	Porta 1521, 8080, 5173, 8180 ou 8085 em uso por outro programa.	Encerre o programa ou altere a porta no `.env`. Ao mudar `FRONTEND_PORT` ou `KEYCLOAK_PORT`, ajuste também o client no Keycloak (seção 8.3).
Login em laço ou erro `Invalid parameter: redirect_uri`	Endereço de retorno não cadastrado no client `datagis-frontend` (porta ou domínio diferente de `http://localhost:5173`).	Cadastre o endereço em Valid redirect URIs e Web origins do client, pelo console do Keycloak.
Todas as chamadas da API retornam 401 mesmo logado	`iss` do token diferente da URL pública configurada (`KEYCLOAK_URL_PUBLICA`).	Acesse o Keycloak pela mesma URL configurada no backend (por exemplo `localhost`, não `127.0.0.1`) e reinicie o backend.
`curl /api/fontes` retorna 401	Comportamento esperado: a API exige token.	Obtenha um token (seção 10.1) ou use o botão Authorize do Swagger.

Sintoma	Causa provável	Solução
Tela "Sem acesso liberado" após o login	Usuário sem perfil ou permissões no Keycloak.	Um administrador deve atribuir um perfil na tela Controle de Acesso > Usuários. O usuário precisa sair e entrar novamente.
Mapa de Indicadores sem camadas	GeoServer não configurado ou indicadores ainda não publicados.	Execute `docker compose run --rm geoserver-config`, confirme em `http://localhost:8085/geoserver` e rode "Migrar para Gold" na tela Lotes e Pipeline.
Tela em branco / "Failed to resolve import" no frontend	Dependências npm desatualizadas.	`docker compose restart frontend`. Se persistir, remova o volume `frontend-node-modules`.
Erro de proxy `ENOTFOUND backend` no Vite	Frontend executado fora do Docker com o alvo `http://backend:8080`.	Altere o alvo do proxy para `http://localhost:8080` (seção 9.3).
Upload termina com "Failed to fetch" e sem erro no log	Conexão encerrada por limite de tamanho no servidor/proxy.	Verifique as propriedades de multipart (seção 7.1) e o `client_max_body_size` do proxy reverso.
Contêiner do backend encerrado por falta de memória	Heap de 8 GB maior que a memória disponível para o Docker.	Aumente a memória do Docker Desktop/WSL ou reduza `JAVA_TOOL_OPTIONS` (ex.: `-Xmx3g`) no `docker-compose.yml`.
Ingestão de fonte remota falha com erro 502	Serviço externo indisponível, lento ou bloqueando a requisição.	Tente novamente mais tarde; confira a URL cadastrada e a mensagem detalhada exibida na tela de fontes.
Botões de executar/upload desabilitados em uma fonte	Mapeamento de colunas incompleto ou sem tabela Silver de destino.	Complete o de-para na tela Cadastro de Fontes (ver Manual do Usuário).
Teste `DatagisApplicationTests` falha dentro do contêiner	Testcontainers sem acesso ao Docker do host.	Execute os testes na máquina (seção 11).

15. Referências

- `CLAUDE.md` (raiz do backend): decisões de arquitetura e histórico técnico do projeto.
- `docs/tutorial-oracle-enterprise-dev-local.md`: passo a passo do acesso à imagem Oracle Enterprise.
- `docs/controle-de-acesso-rbac.md`: perfis, permissões e funcionamento do Keycloak.
- `docs/gold-procedures-calculo-publicacao.md` e `docs/publicacao-gold-paralela.md`: cálculo e publicação dos indicadores.
- `docs/fontes-de-dados.md` e `docs/fontes-de-dados-v2.md`: catálogo e testes das fontes de dados externas.
- `docs/modelagem-silver-gold-quarentena.md`: modelo de dados das camadas.
- `docs/motor-validacao.md`: regras de validação Bronze → Silver.
- `docs/upload-shapefile-car.md`: procedimento de upload de shapefile do CAR.
- Manual do Usuário DataGIS (versão 2.0): operação das telas do sistema.

16. Controle de Versões

Versão	Data	Autor	Descrição
1.0	24/09/2026	Equipe LizardsDBA	Versão inicial do manual de instalação.
2.0	24/10/2026	Equipe LizardsDBA	Atualização para a Sprint 2: autenticação com Keycloak, Oracle Enterprise, GeoServer, ambiente publicado com Caddy, novos schemas e migrations (até V082), novas variáveis de ambiente, verificação com token e novas entradas de solução de problemas.